System design package

Four complementary views of an AI-assisted network intrusion detection system

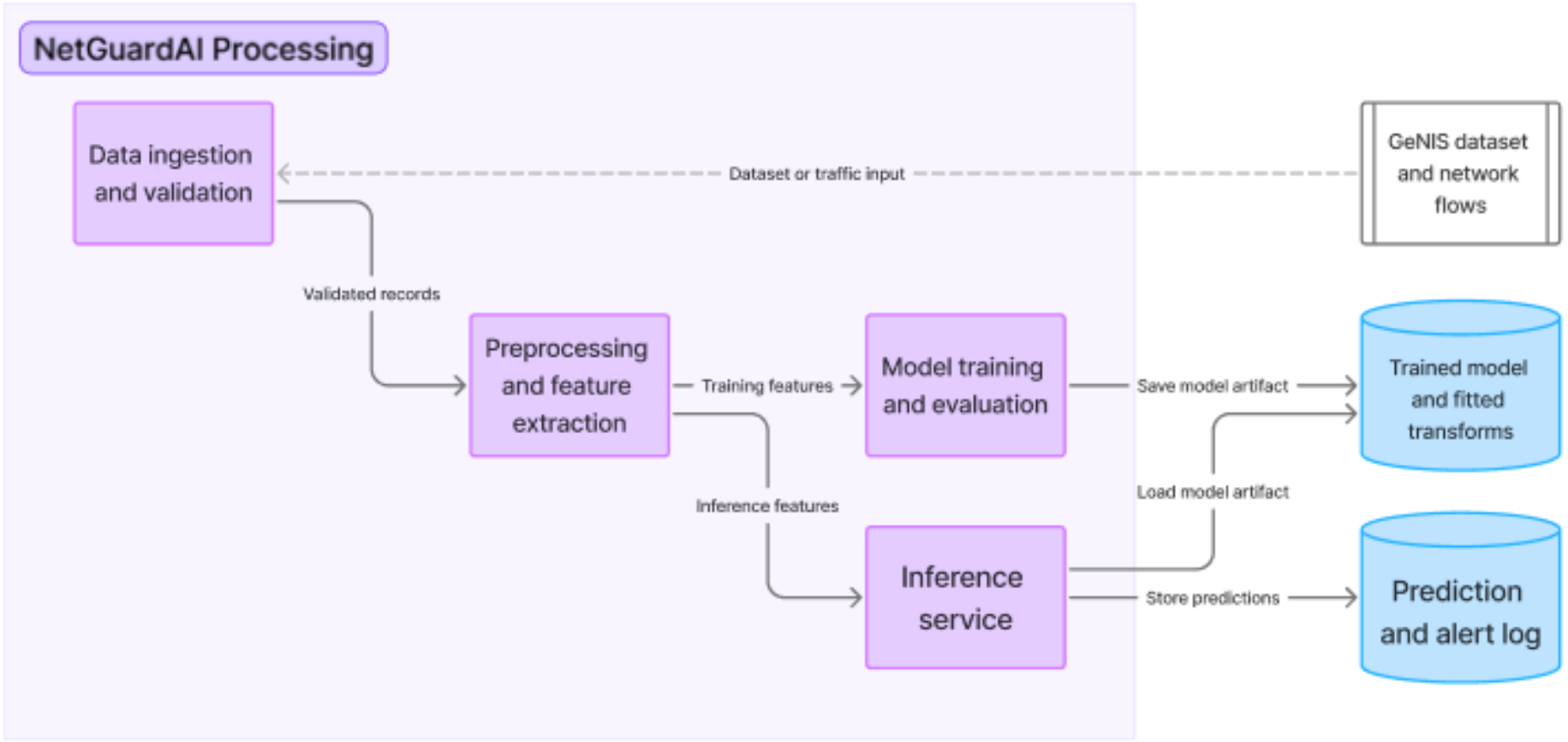
Scope note: The supplied Module 1 notebook covers offline GeNIS data ingestion, preprocessing, class reporting, and Random Forest baseline evaluation. Live traffic capture, inference service, alerting, and dashboard are shown as proposed system-level components; include them in the final design only if they are in your project scope.

Selected designs and their purpose

Component diagram	Defines system modules and how they connect.
Data Flow Diagram	Traces traffic, processes, data stores, and outputs.
Activity diagram	Shows validation and benign-versus-attack logic.
Sequence diagram	Shows message order for one traffic flow.

Design assumptions are stated on each page; proposed features should be updated to match implementation.

NetGuardAI Component Architecture



Why this design was chosen

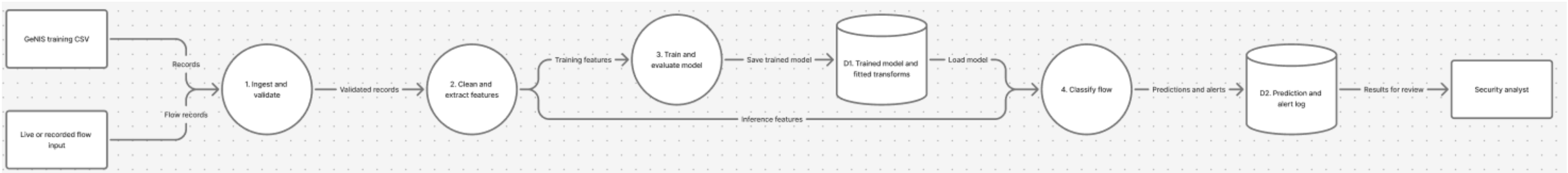
The component diagram gives a compact map of each system responsibility and the connections between them. It makes the offline training path distinct from the proposed inference path.

How it fits Module 1

Module 1 implements data loading, preprocessing, training, and evaluation. Live traffic inference and persistent alert logging are proposed extensions unless already implemented elsewhere.

2. Data Flow Diagram

Exact reference image supplied for NetGuardAI



Why this design was chosen

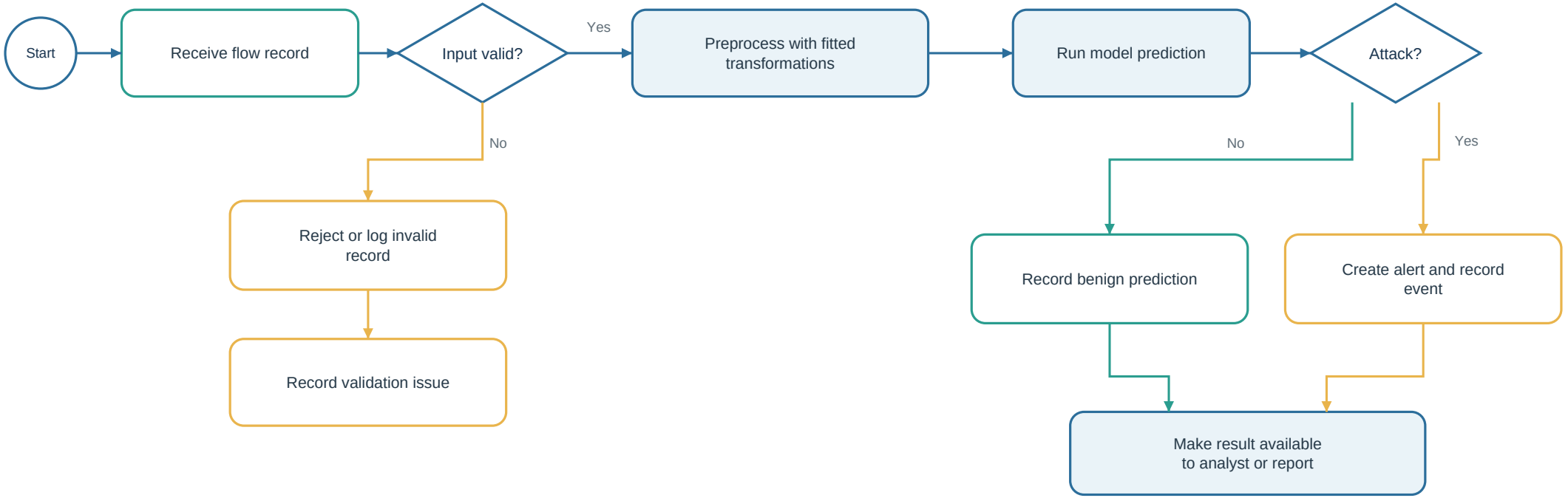
The DFD makes data boundaries visible: where traffic enters, which processes transform it, what data is stored, and who receives the results. It is useful for explaining the separation of training data, model artifacts, predictions, and alerts.

How it fits Module 1

The notebook covers GeNIS dataset ingestion, preprocessing, model training, and evaluation. Live-flow input and analyst-facing alert output are proposed deployment elements unless implemented elsewhere.

3. Activity Diagram

Behavioral design | Detection workflow and decision paths

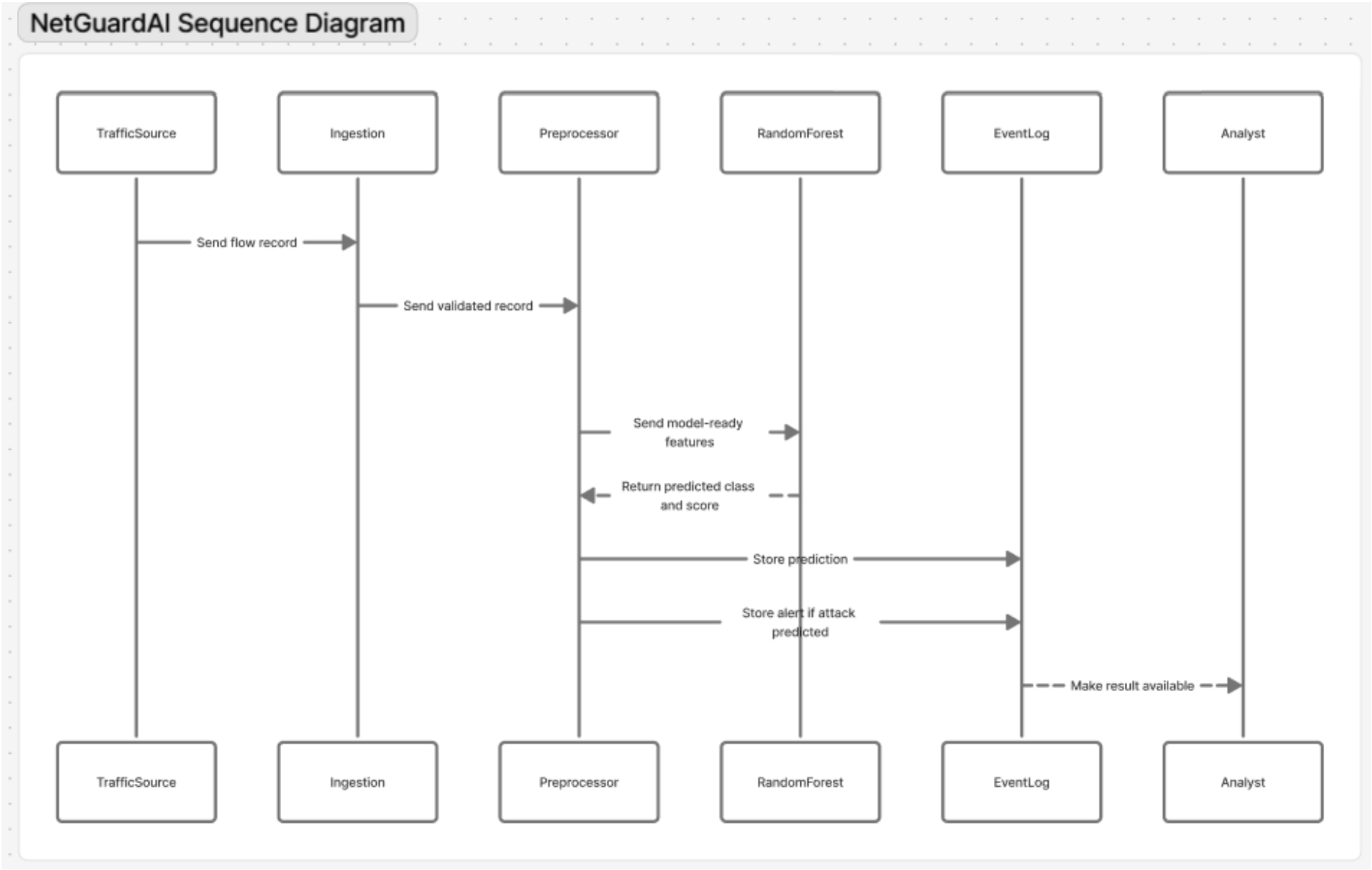


Why this design was chosen

The activity diagram explains control flow. It shows the validation failure path and the two outcomes after classification, so a reader can follow what happens to both invalid and valid traffic.

How it fits Module 1

The notebook implements offline preprocessing and classification. Live input, alert creation, and analyst presentation are proposed runtime behavior unless implemented elsewhere.



Why this design was chosen

The sequence diagram clarifies who sends each message and in what order. It shows the model response returning to preprocessing before predictions are logged and made available to the analyst.

How it fits Module 1

Module 1 trains and evaluates the model offline; this runtime interaction sequence is proposed for a deployed detector.